EECS 281 — Exam 2 Practice Questions

Sorting · Quicksort · Hash Tables · Trees · AVL

Questions tagged **PAST EXAM** (source) are copied word for word from the Fall 2026 Exam 2 practice exam. Questions tagged **NEW – SIMILAR PATTERN** (modeled on source) are new, written to follow the same patterns with different numbers and twists. Answers are on the last pages.

Part I — Multiple Choice

1. Quicksort

PAST EXAM (Practice Exam 2, Q2)

Suppose we run quicksort on the following list $[4, 12, 9, 1, 2, 5, 3, 8, 7]$. We choose the last element, 7, as the pivot. After partitioning as described in lecture (but **before** we run the recursive function calls), what is the resulting list?

A) $[4, 3, 5, 1, 2, 7, 12, 9, 8]$
B) $[4, 3, 5, 1, 2, 9, 12, 8, 7]$
C) $[4, 3, 1, 5, 2, 7, 12, 8, 9]$
D) $[4, 3, 5, 1, 2, 7, 12, 8, 9]$

2. Quicksort Partition

NEW – SIMILAR PATTERN (modeled on Practice Exam Q2)

Suppose we run quicksort on the array $[8, 3, 14, 6, 11, 2, 9, 1, 13, 7]$, choosing the last element, 7, as the pivot. After one call to `partition()` as described in lecture (code below, called on the range `[0, 10)`), but **before** the recursive calls, what is the resulting array?

```
1  int32_t partition(std::vector<int32_t>& vec, int32_t left, int32_t right) {
2    int32_t pivot = --right;
3    while (true) {
4      while (vec[left] < vec[pivot]) ++left;
5      while (left < right && vec[right - 1] >= vec[pivot]) --right;
6      if (left >= right) break;
7      std::swap(vec[left], vec[right - 1]);
8    } // while
9    std::swap(vec[left], vec[pivot]);
10   return left;
11 } // partition()
```

A) $[3, 6, 2, 1, 7, 14, 9, 8, 13, 11]$
B) $[1, 3, 2, 6, 7, 14, 9, 8, 13, 11]$
C) $[1, 3, 2, 6, 7, 11, 14, 9, 8, 13]$
D) $[3, 6, 2, 1, 7, 8, 14, 11, 9, 13]$

3. Sorting

NEW – SIMILAR PATTERN (modeled on Practice Exam Q1 & Q11)

Which of the following statements is **true**?

A) Heapsort is stable, and its worst-case time complexity is $\Theta(n \log n)$.
B) If every element of an array is at most k positions away from its final sorted position, insertion sort runs in $O(nk)$ time on that array.
C) Quicksort as described in lecture (last element as pivot) runs in $\Theta(n \log n)$ time on an array that is already sorted.
D) Mergesort is adaptive: on an already-sorted array it runs in $\Theta(n)$ time.
E) Because insertion sort runs in $\Theta(n)$ on sorted input, it is possible to design a comparison sort whose worst-case time on arbitrary input is $\Theta(n)$.

4. Collision Resolution PAST EXAM (Practice Exam 2, Q4)

You are given a hash table with size $M = 10$ and hash function $H(n) = (2n+3) \bmod M$. Collisions are resolved using quadratic probing. What would the table look like (with an X denoting an empty location) after inserting the following elements in the order given? Use $c_1 = 0$ and $c_2 = 1$.

$$4,\ 10,\ 12,\ 7,\ 2$$

A) $[X, 4, X, 10, X, X, X, 12, 7, 2]$
B) $[X, 4, 2, 10, X, X, X, 12, 7, X]$
C) $[X, 4, X, 10, X, X, 2, 12, 7, X]$
D) $[X, 4, X, 10, X, X, 2, 12, X, 7]$

5. Collision Resolution NEW – SIMILAR PATTERN (modeled on Practice Exam Q4)

A hash table has size $M = 11$ and primary hash function $t(k) = k \bmod 11$. Collisions are resolved with double hashing: the j-th probe ($j = 0, 1, 2, \ldots$) examines index $(t(k) + j \cdot t'(k)) \bmod 11$, where $t'(k) = 7 - (k \bmod 7)$. Erasing a key replaces it with a *deleted* marker (tombstone). Duplicates are not allowed. To insert, probe as in a search (continuing past deleted slots) until the key is found (no insertion) or an empty slot is reached. Then place the key in the *first* deleted or empty slot seen. Perform these operations in order:

$$\text{insert } 35,\ 40,\ 24,\ 72,\ 79; \quad \text{erase } 35; \quad \text{insert } 24; \quad \text{insert } 34$$

What does the table look like afterward? (X = empty, DEL = deleted.)

A) `[X, 34, DEL, 24, 79, X, 72, 40, X, X, X]`
B) `[72, 79, DEL, 34, X, X, 24, 40, X, X, X]`
C) `[72, 79, 34, X, X, X, 24, 40, X, X, X]`
D) `[72, 79, 24, 34, X, X, 24, 40, X, X, X]`

6. Tree ADT PAST EXAM (Practice Exam 2, Q7)

What does the following function return?

```
1  int count(Node* curNode) {
2    if (curNode == nullptr)
3      return 0;
4    else if (curNode->parent == nullptr)
5      return 1 + count(curNode->left) + count(curNode->right);
6    else if (curNode->left == nullptr && curNode->right == nullptr)
7      return 1;
8    else
9      return count(curNode->left) + count(curNode->right);
10 } // count()
```

A) The number of nodes in the tree
B) The number of internal nodes
C) The number of external (leaf) nodes
D) The number of external (leaf) nodes plus the root
E) The depth of the tree

7. Tree ADT NEW – SIMILAR PATTERN (modeled on Practice Exam Q7)

Using `struct Node { int val; Node *left; Node *right; };`, what does `mystery(root, k)` return for $k \geq 0$? The root is at depth 0.

```
int mystery(Node *node, int k) {
  if (node == nullptr)
    return 0;
  if (k == 0)
    return (node->left == nullptr && node->right == nullptr) ? 1 : 0;
  return mystery(node->left, k - 1) + mystery(node->right, k - 1);
} // mystery()
```

A) The number of nodes at depth exactly k
B) The number of leaf nodes at depth at most k
C) The number of leaf nodes at depth exactly k
D) The number of internal (non-leaf) nodes at depth exactly k
E) The number of leaf nodes in the entire tree, regardless of k

8. Tree Traversal PAST EXAM (Practice Exam 2, Q8)

Given the following preorder and inorder traversals of a binary tree, what is the postorder traversal of this same binary tree?

- Preorder: 11, 19, 31, 7, 10
- Inorder: 31, 19, 11, 10, 7

A) 11, 19, 7, 31, 10
B) 31, 19, 10, 7, 11
C) 19, 31, 10, 7, 11
D) 31, 10, 7, 19, 11
E) 10, 7, 31, 19, 11

9. Tree Traversal NEW – SIMILAR PATTERN (modeled on Practice Exam Q8)

Given the following postorder and inorder traversals of a binary tree, what is the *preorder* traversal of the same tree?

- Postorder: 27, 15, 9, 4, 21, 6, 30, 12
- Inorder: 27, 4, 15, 9, 12, 30, 21, 6

A) 12, 4, 27, 9, 15, 30, 6, 21
B) 12, 30, 6, 21, 4, 9, 15, 27
C) 12, 4, 27, 9, 15, 30, 21, 6
D) 12, 4, 30, 27, 9, 6, 15, 21
E) 12, 4, 27, 15, 9, 30, 21, 6

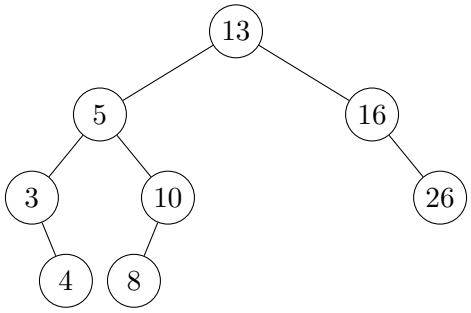
10. AVL Trees PAST EXAM (Practice Exam 2, Q10)

Which of the following statements about AVL trees is **true**?

A) Optimally re-balancing an AVL tree after a deletion can take more than $O(1)$ time.
B) For an insert operation, up to three rotations are needed.
C) Searching an AVL tree can take longer than $O(\log n)$.
D) Given an n-node binary search tree that has never been balanced, the BST can be made into a balanced AVL tree using $O(\log n)$ rotations.
E) A tree is AVL balanced if and only if the absolute difference in heights of the root's children is less than 2.

11. Insertion/Deletion PAST EXAM (Practice Exam 2, Q12)

Given the following AVL tree:



Perform the following operations:

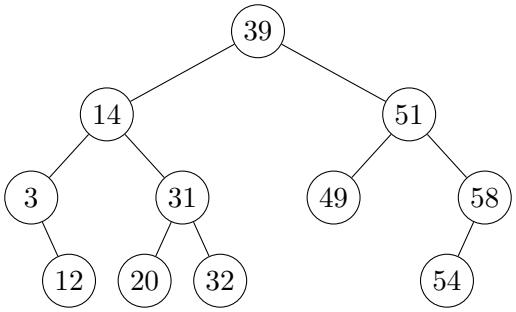
1. Delete 13 (replace with its inorder successor)
2. Insert 9

What are the left and right children of "16" after performing the operations?

A) 8, 26
B) 9, 26
C) 10, 26
D) `nullptr`, 26
E) `nullptr`, `nullptr`

12. Insertion/Deletion NEW – SIMILAR PATTERN (modeled on Practice Exam Q12)

Given the following AVL tree:



Perform the following operations, rebalancing with AVL rotations after each one:

1. Delete 39 (replace it with its inorder successor)
2. Insert 33

What are the left and right children of "49" after performing the operations?

A) 14, 51
B) 14, 54
C) 33, `nullptr`
D) 32, 54
E) 14, 58

Part II — Written

13. Programming: Counting in a Range [8 Points] NEW – SIMILAR PATTERN (modeled on Practice Exam Q13)

Using the following definition of a binary tree node:

```
struct Node {
  int val;
  Node *left;
  Node *right;
};
```

`node` is the root of a binary search tree with distinct values. Implement `countInRange()`, which returns the number of values v in the tree with `lo` $\leq v \leq$ `hi`. You may assume `lo` $\leq$ `hi`.

Requirements: Your solution must run in $O(h+k)$ time, where h is the height of the tree and k is the value returned. You may use up to $O(h)$ auxiliary space. (A solution that visits every node, $\Theta(n)$, will not receive full credit.)

Implementation: Limit: 10 lines of code. You may use any C, C++, or STL function/algorithm.

```
int countInRange(Node *node, int lo, int hi) {
```

14. Programming: Inserting into a Hash Table [12 Points] NEW – SIMILAR PATTERN (modeled on Practice Exam Q14)

Given the following definition of a hash table structure:

```
enum class Status { Empty, Occupied, Deleted };
struct HashTable {
  std::vector<int> buckets;
  std::vector<Status> status;
};
```

The table resolves collisions with quadratic probing. Let M be the table size (`buckets.size()`) and $h($`val`$) =$ `val` mod M. You may assume `val` ≥ 0. The i-th probe ($i = 0, 1, \ldots, M-1$) examines index $(h($`val`$) + c_1 i + c_2 i^2)$ mod M. Use $c_1 = 2$ and $c_2 = 1$. Erased values are marked `Deleted` (tombstones), and the set allows no duplicates.

Implement `insert()`. If `val` is already in the table, return `false` without changing the table. Otherwise, store `val` in the *first* `Deleted` or `Empty` slot encountered while probing, mark it `Occupied`, and return `true`. As in lecture, a `Deleted` slot does *not* end the search, but an `Empty` slot does. If the M probes reach no `Empty` slot, do not find `val`, and pass no `Deleted` slot, return `false`.

Requirements: Your solution must implement quadratic probing. It must run in $O(M)$ time and use $O(1)$ auxiliary space.

Implementation: Limit: 20 lines of code. You may use any C, C++, or STL function/algorithm.

```
bool insert(HashTable &tbl, int val) {
```

Answer Key

1. Quicksort (past) Answer: **D**

Pivot = 7. `L` moves right while $v[L] < 7$; `R` moves left while $v[R-1] \geq 7$; swap; repeat.

- `L` stops at 12, `R` stops at 3 $\Rightarrow [4, 3, 9, 1, 2, 5, 12, 8, 7]$
- `L` stops at 9, `R` stops at 5 $\Rightarrow [4, 3, 5, 1, 2, 9, 12, 8, 7]$
- `L` stops at 9 (index 5), `R` meets `L` $\Rightarrow$ swap 9 with the pivot: $[4, 3, 5, 1, 2, 7, 12, 8, 9]$

2. Quicksort Partition (new) Answer: **B**

- `L` stops at 8 (index 0); `R` skips 13 and stops at 1 $\Rightarrow [1, 3, 14, 6, 11, 2, 9, 8, 13, 7]$
- `L` stops at 14 (index 2); `R` skips 8, 9 and stops at 2 $\Rightarrow [1, 3, 2, 6, 11, 14, 9, 8, 13, 7]$
- `L` stops at 11 (index 4); `R` skips 14, 11 and meets `L` $\Rightarrow$ swap 11 with pivot: $[1, 3, 2, 6, 7, 14, 9, 8, 13, 11]$

Distractors: A is Lomuto partitioning, C shifts the pivot into place instead of swapping, and D is a stable partition.

3. Sorting (new) Answer: **B**

For insertion sort, each element only has to shift past larger elements that started before it. All of those lie within $2k$ positions, so the work is $O(k)$ per element and $O(nk)$ total. (A) Heapsort is *not* stable. (C) On sorted input the last element is always the max, so the splits are $n-1$ and 0, giving $\Theta(n^2)$. (D) Mergesort is $\Theta(n \log n)$ in every case. (E) Comparison sorts need $\Omega(n \log n)$ comparisons in the worst case, since $\log_2(n!)$ comparisons are needed.

4. Collision Resolution (past) Answer: **C**

Probe index = $(H(n)+i^2) \bmod 10$. $4 \to 11 \bmod 10 = 1$. $10 \to 3$. $12 \to 7$. $7 \to 7$ (taken), $7+1 = 8$. $2 \to 7$ (taken), 8 (taken), $7+4 = 11 \to 1$ (taken), $7+9 = 16 \to 6$. Result: $[X, 4, X, 10, X, X, 2, 12, 7, X]$.

5. Collision Resolution (new) Answer: **C**

$35 \to 2$. $40 \to 7$. $24 \to 2$ (taken), $t' = 4$: slot 6. $72 \to 6$ (taken), $t' = 5$: $11 \bmod 11 = 0$. $79 \to 2$ (taken), $t' = 5$: 7 (taken), $12 \bmod 11 = 1$. Erase 35: slot 2 becomes DEL. Insert 24: slot 2 is DEL, so keep probing; 24 is found at slot 6, so nothing changes. Insert 34: $34 \bmod 11 = 1$ (taken), $t'(34) = 1$: slot 2 is DEL (remember it), slot 3 is empty, so 34 is absent. Place it in the first DEL slot, slot 2. Result: `[72, 79, 34, X, X, X, 24, 40, X, X, X]`. Distractors: A uses linear probing, B does not reuse the tombstone, and D stops the search at DEL, which inserts a duplicate 24.

6. Tree ADT (past) Answer: **D**

A leaf returns 1. A non-root internal node adds nothing of its own; it only sums its children. The root adds 1 on top of its subtrees. So the function counts the leaves, plus one for the root.

7. Tree ADT (new) Answer: **C**

Each call moves one level down and decreases k, so k reaches 0 exactly at depth k. Only those nodes can contribute, and only if they are leaves. Shallower nodes add nothing (there is no "1+"), and the recursion never goes deeper than k.

8. Tree Traversal (past) Answer: **B**

The root is 11. Inorder splits into left $\{31, 19\}$ and right $\{10, 7\}$. The left preorder is $19, 31$, so 19 has left child 31. The right preorder is $7, 10$, so 7 has left child 10. Postorder: $31, 19, 10, 7, 11$.

9. Tree Traversal (new) Answer: **A**

The last postorder entry, 12, is the root. Inorder gives left $\{27, 4, 15, 9\}$ and right $\{30, 21, 6\}$. Left: its postorder is $27, 15, 9, 4$, so 4 is the root, with left child 27 and right child 9, and 9 has left child 15.

Right: its postorder is $21, 6, 30$, so 30 is the root, with right child 6, and 6 has left child 21. Preorder: $12, 4, 27, 9, 15, 30, 6, 21$. Distractors: B is the reversed postorder, D is the level order, and C and E use the *first* postorder element as the root.

10. AVL Trees (past) Answer: **A**

A deletion can trigger a rotation at every level on the way up, i.e. $O(\log n)$ rotations. (B) An insert needs at most one single or one double rotation. (C) AVL height is $\Theta(\log n)$, so search is $O(\log n)$. (D) A degenerate chain needs $\Omega(n)$ rotations. (E) *Every* node must be balanced, not just the root.

11. Insertion/Deletion (past) Answer: **B**

Delete 13: the successor 16 moves to the root and 26 takes 16's old spot. The root is now $+2$ and its left child 5 has balance 0, so do a *single* right rotation. Result: $5\big(3(\varnothing, 4),\ 16(10(8, \varnothing),\ 26)\big)$.
Insert 9: 9 becomes 8's right child. Node 10 is $+2$ with child 8 at -1 (the LR case), so do a double rotation: 9 replaces 10, with children 8 and 10. 16's children are now **9 and 26**. (A double rotation on the delete step, which is wrong when the child's balance is 0, leads to D.)

12. Insertion/Deletion (new) Answer: **D**

Delete 39: the successor 49 (a leaf) moves to the root. Now 51 has balance -2, and its right child 58 is $+1$ (the RL case): rotate right at 58, then left at 51, giving $54(51, 58)$. The tree is $49\big(14(3(\varnothing, 12),\ 31(20, 32)),\ 54(51, 58)\big)$.
Insert 33: 33 becomes 32's right child. The root 49 is now $+2$ (left height 3 vs. right height 1), and its left child 14 is -1 (the LR case). Rotate left at 14, then right at 49. The final tree is $31\big(14(3(\varnothing, 12), 20),\ 49(32(\varnothing, 33),\ 54(51, 58))\big)$, so 49's children are **32 and 54**. Distractors: A skips rebalancing entirely, B skips rebalancing after the insert, C uses the predecessor, and E uses a single rotation on the delete.

13. Counting in a Range (new) Answer: **reference solution**

```
int countInRange(Node *node, int lo, int hi) {
  if (node == nullptr) return 0;
  if (node->val < lo) return countInRange(node->right, lo, hi); // prune left
  if (node->val > hi) return countInRange(node->left, lo, hi);  // prune right
  return 1 + countInRange(node->left, lo, hi) + countInRange(node->right, lo, hi);
} // countInRange()
```

Every visited node is either in range (at most k of them) or on the search path for `lo` or for `hi` (at most $2(h+1)$ of them). That gives $O(h+k)$ time, plus $O(h)$ recursion stack. Common deductions: a full traversal; recursing into both children when the node is out of range; off-by-one errors with `<` vs. `<=`.

14. Inserting into a Hash Table (new) Answer: **reference solution**

```
bool insert(HashTable &tbl, int val) {
  size_t M = tbl.buckets.size(), home = static_cast<size_t>(val) % M;
  size_t target = M;  // first Deleted/Empty slot seen (M = none yet)
  for (size_t i = 0; i < M; ++i) {
    size_t idx = (home + 2 * i + i * i) % M;  // c1 = 2, c2 = 1
    if (tbl.status[idx] == Status::Empty) {
      if (target == M) target = idx;
      break;  // Empty ends the search: val is not in the table
    }
    if (tbl.status[idx] == Status::Deleted) {
      if (target == M) target = idx;  // remember it, but keep probing
    } else if (tbl.buckets[idx] == val) {
      return false;  // already present
    }
  }
  if (target == M) return false;  // no usable slot on the probe sequence
  tbl.buckets[target] = val;
  tbl.status[target] = Status::Occupied;
  return true;
} // insert()
```

Two traps. (1) Stopping at the first `Deleted` slot: `val` may be further along the probe sequence, so this creates duplicates. (2) Not reusing tombstones: the key must go in the *first* `Deleted` or `Empty` slot. Also, never compare `buckets[idx]` on a `Deleted` slot, because the stale value is still stored there. There are at most M probes with $O(1)$ work each, so time is $O(M)$ and extra space is $O(1)$.